



A probabilistic greedy search algorithm for combinatorial optimisation with application to the set covering problem

M Haouari* and JS Chaouachi

Ecole Polytechnique de Tunisie, La Marsa, Tunisia

We present a probabilistic greedy search method for combinatorial optimisation problems. This approach is implemented and evaluated for the Set Covering Problem (SCP) and shown to yield a simple, robust, and quite fast heuristic. Tests performed on a large set of benchmark instances with up to 1000 rows and 10 000 columns show that the algorithm consistently yields near-optimal solutions.

Journal of the Operational Research Society (2002) 53, 1–8. DOI: 10.1057/palgrave/jors/2601366

Keywords: combinatorial optimisation; greedy method; heuristics; set covering problem

Introduction

A combinatorial optimisation problem might be defined as follows (Nemhauser and Wolsey¹):

Let $N = \{1, \dots, n\}$ be a finite set and let $c = \{c_1, \dots, c_n\}$ be a weight n -vector. For a subset $F \subseteq N$ define the weight of F as $c(F) = \sum_{j \in F} c_j$. Suppose we are given a collection of feasible subsets Φ of N . The combinatorial optimisation problem is:

$$\text{Minimize } \{c(F) : F \in \Phi\}$$

Within the wide range of exact and heuristic algorithms presented in the operational research literature for solving combinatorial optimisation problems, the most natural approach is the greedy algorithm. The principle of the greedy algorithm is elementary and widely applicable: it builds up a solution by making the best choice locally. More precisely, it starts with $F = \{i\}$ where $i \in N$ and c_i is minimal. Then, the greedy method iteratively adds to F an element $j \in N \setminus F$ such that the weight $c(F \cup \{j\})$ is minimal. Once an element is chosen, it is kept permanently in the solution and the algorithm terminates when a feasible solution is obtained (ie $F \in \Phi$). Perhaps the most important example of a combinatorial optimisation problem where this simple idea works is the minimum spanning tree problem which is defined as follows: Given a general connected undirected graph $G = (V, E)$ with a node set V and an edge set E , a spanning tree in G is a connected subgraph $T = (V, E')$ containing no cycles.

Given a weight function $c : E \rightarrow \mathbb{Z}_+$, (where $\mathbb{Z}_+$ denotes the set of integers), a minimal weight spanning tree is a spanning tree T^* for which $\sum_{e \in T^*} c_e$ is minimal. A greedy solution procedure (known as *Kruskal's algorithm*) constructs a minimal weight spanning tree by examining the edges one at a time in non-decreasing order of weight and rejecting an edge if it forms a cycle with those already chosen.

However, for most combinatorial problems we cannot expect the greedy algorithm to yield an optimal solution (in other words, in general greed does not pay). Nevertheless, we may still use a greedy algorithm as a heuristic for a quick solving of hard problems. Actually, there are two main reasons that make heuristics based on the greedy principle very popular in operational research. Firstly, they are intuitive and simple to grasp. Secondly, they are generally quite fast in execution and easy-to-code. However, unfortunately a common drawback of greedy heuristics is that they are not robust, and may provide very poor quality solutions. For instance, in our experimentations with the Set Covering Problem (SCP), we have found that it provides solutions that are roughly 12% in excess of those obtained with the best heuristics. In the same vein, Johnson *et al*² found that a greedy algorithm for the Travelling Salesman Problem (TSP), known as *the multiple fragments heuristic*, typically provides a deviation ranging between 14 and 19%.

The aim of this paper is to develop an improved general purpose greedy-type approach that keeps as much as possible all the desirable characteristics of the standard greedy method while significantly improving its robustness and quality performance. Basically, our approach is an iterated probabilistic greedy method which incorporates diversification schemes along with a learning strategy. Clearly, more powerful special purpose algorithms that are superior in

*Correspondence: M Haouari, Laboratory of Mathematical Engineering, Ecole Polytechnique de Tunisie, BP 743, 2080 La Marsa, Tunisia.
E-mail: mohamed.haouari@ept.rnu.tn

accuracy may be available, but the simplicity and versatility of our approach might make it an attractive alternative. It is worth noting that our method shares some common aspects with the well-known *Greedy Randomized Search Procedure* (GRASP) (Feo and Resende^{3,4}), but it exhibits several important differences that are discussed in the section title ‘The probabilistic greedy search method’.

Although, our method is easily applicable for a wide range of hard combinatorial optimisation problems, for the sake of clarity, we will focus solely on the description of its implementation for the SCP. This is an extremely important prototype combinatorial problem, which is extensively investigated in the discrete optimisation literature. The paper is organized as follows. In the following section, we present a succinct review of the SCP. In the subsequent sections, we discuss the basic components of our probabilistic greedy approach as tailored for the SCP, followed by presentation of experimental results. Finally, we provide some concluding remarks.

The set covering problem

The SCP might be described as follows: Given a set S of m objects, and a family $\mathcal{F} = \{S_1, \dots, S_n\}$ where $S_j \subseteq S$ ($j = 1, \dots, n$), together with a non-negative cost c_j which is defined for each subset $S_j \in \mathcal{F}$, a *cover* is a family $\mathcal{C} \subseteq \mathcal{F}$ such that $S = \bigcup_{k: S_k \in \mathcal{C}} S_k$. The cost of a cover $\mathcal{C}$ is $\sum_{k: S_k \in \mathcal{C}} c_k$. The SCP is to find a cover $\mathcal{C}^*$ of minimal cost.

The SCP can be modelled as an integer program by defining for each index j ($j = 1, \dots, n$):

- x_j as a binary variable which equals 1 if $S_j \in \mathcal{C}^*$, and 0 otherwise
- a_{ij} as a binary constant which equals 1 if object $i \in S_j$ ($i = 1, \dots, m$), and 0 otherwise.

The mathematical formulation is:

$$\text{Min} \sum_{j=1}^n c_j x_j \quad (1)$$

$$\sum_{j=1}^n a_{ij} x_j \geq 1 \quad i = 1, \dots, m \quad (2)$$

$$x_j \in \{0, 1\} \quad j = 1, \dots, n \quad (3)$$

This formulation suggests that the SCP can be viewed as the problem of selecting a minimum cost subset of columns of a 0–1 matrix such that for every row i ($i = 1, \dots, m$) there is at least one selected column such that $a_{ij} = 1$.

Despite its deceptive simplicity, the SCP belongs to the class of $\mathcal{NP}$ -hard problems and is considered as one of the most fundamental combinatorial optimisation problems. Its theoretical importance stems from its versatility since it generalizes several important combinatorial optimisation problems including node (and edge) covering, set partition-

ing, set packing, and min-cut. Moreover, the SCP is commonly used for modelling a wide range of practical applications. A partial list includes airline crew scheduling (Hoffman and Padberg⁵), bus driver scheduling (Paia and Paixao⁶), vehicle routing (Desrosiers *et al*⁷), and facility location (Current and O’Kelly⁸).

Many algorithms have been developed so far for solving the SCP. Most exact approaches are branch-and-bound and branch-and-cut methods (see Beasley,⁹ Beasley and Jörnsten,¹⁰ El-Darzi and Mitra,¹¹ Nobili and Sassano,¹² Rushmeier and Nemhauser,¹³ Harche and Thompson,¹⁴ Balas and Carrera,¹⁵ Balas *et al*¹⁶). These exact methods require a substantial computational effort to solve instances with a few hundred rows and a few thousand columns, particularly those with high density (ie large fraction of non-zero entries). However, several practical problems involve much larger instances and, in this case, heuristic algorithms constitute the only viable alternative solution strategy. The earliest (and the simplest) such approach is the greedy algorithm of Chvátal.¹⁷ Recently, very effective heuristics have been designed for tackling the SCP. The most effective ones are based on Lagrangian relaxation (Beasley,¹⁸ Beasley and Jörnsten,¹⁰ Lorena and Lopes,¹⁹ Ceria *et al*,²⁰ Caprara *et al*²¹) and local search (Jacobs and Brusco,²² Beasley and Chu,²³ Al-Sultan *et al*,²⁴ Lorena and Lopes²⁵).

The probabilistic greedy search method

In this section, we develop a PRObabilistic GREedy Search (PROGRES) method. Basically, PROGRES is an iterative procedure which consists in repeatedly running a greedy construction method which is similar to the one discussed by Chvátal.¹⁷

First we introduce some additional notation:

$A = (a_{ij})$:	0–1 matrix whose columns are $A_1, \dots, A_n$, respectively
I	: set of rows of A
J	: set of columns of A
I_j	: set of rows such that $a_{ij} = 1$
C	: set of columns in a solution
U	: set of uncovered rows
α_j	: number of rows in U such that $a_{ij} = 1$
β_i	: number of columns in C that cover row i

We assume w.l.g. that the columns of A are indexed by non-decreasing cost.

Initially, $\mathcal{C}$ is empty. The greedy method consists in iteratively adding to $\mathcal{C}$ a column which minimises some selection criteria, and removing from $\mathcal{C}$ the redundant columns. The algorithm stops when all the rows are covered by the columns in $\mathcal{C}$. The selection criterion which we found the most effective is the following:

$$f_j = \frac{c_j}{\alpha_j}$$

A formal description of the greedy method is the following:

Greedy Algorithm

Step 1: Initialisation

$$\mathcal{C} = \emptyset, U = I, \beta_i = 0 \forall i \in I$$

Step 2: Column selection

- 2.1 Compute for all $A_j \in J$ the ratio: $f_j = c_j / \alpha_j$
- 2.2 Set $\mathcal{C} = \mathcal{C} \cup \{A_k\}$ where $A_k = \arg(\min_{j: A_j \in J} f_j)$
- 2.3 Set $\beta_i = \beta_i + 1$ for all $i \in I_k$
- 2.4 Set $U = U \setminus I_k$

Step 3: Elimination of redundant columns

Examine all columns of $\mathcal{C}$ by non-increasing order of costs
For all $A_j \in \mathcal{C}$ do (if $\beta_i \geq 2$ for all $i \in I_j$ then remove A_j from $\mathcal{C}$)

Step 4: Termination test

If ($U \neq \emptyset$) then go to step 2 else output $\mathcal{C}$ as the solution

The main computational effort is the sorting of the columns which requires an $O(n \log n)$ time. Therefore, it is extremely fast and can solve large instances within a moderate CPU time.

In order to favour the generation of improved solutions, three diversification techniques are jointly implemented. These techniques are: randomisation, perturbation, and penalisation. Moreover, the approach is accelerated through the application of a pre-processing algorithm. The details of the different components of the PROGRES method are described in the following subsections.

Randomisation

The greedy algorithm can be randomised in the following way: instead of selecting at each iteration the column with a smallest f_j (cf. step 2.2), one may proceed in two steps. Firstly, a set $B = \{A_{(k)} : k = 1, \dots, s\}$ constituted with the s best candidate columns is computed (ties are broken arbitrarily). Secondly, a column in B is randomly selected and appended to $\mathcal{C}$. The selection probability of a column $A_{(k)} \in B$ is:

$$\pi_{(k)} = \frac{c_{\max} - c_{(k)} + 1}{\sum_{j=1,s} (c_{\max} - c_{(j)} + 1)}$$

where $c_{\max} = \max_{j=1,\dots,s} c_{(j)}$. This probability distribution is decreasing with the cost of the columns, and in this way, the columns with the smallest costs are favoured. Hence, some diversification is introduced through the randomisation of the selection process. Obviously, the parameter s should be fixed at a value strictly larger than 1, otherwise the procedure corresponds to the (deterministic) greedy algorithm. A large value of s tends to yield a pure random solution. A formal description of a randomised greedy method is the following:

probabilistic_greedy (s, c, U)

{the input value of U is either the empty set or a non-empty subset of rows}

Step 1: Initialisation

$\mathcal{C}$ = set of columns covering the rows in $I \setminus U$

$$\beta_i = \sum_{j: A_j \in \mathcal{C}} a_{ij} \forall i \in I$$

Step 2: Column selection

- 2.1 Compute for all $A_j \in J$ the ratio: $f_j = c_j / \alpha_j$
- 2.2 Set B = set of the s best columns and compute π_j for each $A_j \in B$
- 2.3 Set $\mathcal{C} = \mathcal{C} \cup \{A_k\}$ where A_k is selected randomly according to the distribution (π_j)
- 2.4 Set $\beta_i = \beta_i + 1$ for all $i \in I_k$
- 2.5 Set $U = U \setminus I_k$

Step 3: Elimination of redundant columns

Examine all columns of $\mathcal{C}$ by non-increasing order of costs

For all $A_j \in \mathcal{C}$ do (if $\beta_i \geq 2$ for all $i \in I_j$ then remove A_j from $\mathcal{C}$)

Step 4: Termination test

If ($U \neq \emptyset$) then go to step 2 else output $\mathcal{C}$ as the solution

It is worth noting that the idea of randomising the greedy procedure goes back to the work of Feo and Resende³ who developed the GRASP meta-heuristic that proved to be effective for a wide range of combinatorial optimisation problems (see Resende²⁶). Roughly speaking, GRASP is a multi-start local search method. At each iteration, a new starting solution is obtained through a probabilistic greedy method. For that purpose, in the context of the SCP, a column is randomly selected according to the uniform distribution, from the best 100 θ th percentile ($\theta \in [0, 1]$ is a given parameter). This construction phase is followed by a local optimisation procedure.

Perturbation

Given a SCP instance P , a perturbed instance $\tilde{P}$ is obtained by slightly modifying the genuine costs of the columns. More precisely, the cost c_j of each column $A_j \in J$ is replaced by $\tilde{c}_j = (1 + \varepsilon_j)c_j$, where ε_j is a small random number (noise), such that $E(\varepsilon_j) = 0$. For instance, in our experiments, ε_j is randomly drawn from the uniform distribution $U[-0.05, 0.05]$. The idea of perturbation implicitly assumes that the optimal solution is not sensitive to a small variation of the data. However, iterating the greedy algorithm with different perturbed costs may lead to different near-optimal solutions. Hence, new regions of the search space will be explored. Consequently, it results in diversifying the search. It is worth noting that the perturbation technique was successfully used in the context of *local search methods* by Storer *et al.*,²⁷ Charon and Hudry,²⁸ and Codenotti *et al.*²⁹

Penalisation

A greedy algorithm that is based only on perturbation and randomisation, lacks the desirable property of self-adaptation. Hence, it is incapable of learning from previously generated solutions. In order to overcome this drawback we implemented a penalisation strategy inspired from a meta-heuristic known as Guided Local Search (Tsang and Voudouris³⁰). This penalisation procedure proceeds in the following way: at the end of each iteration, the columns with the highest cost in the feasible solution are identified, and the cost of these columns is modified by adding a set of penalty terms. The next iteration of the greedy procedure is carried out with the augmented costs. In this way, the greedy procedure is discouraged from generating too frequently the same (costly) columns. The aim of this strategy, is to lead the search towards unexplored regions of the solution space. More precisely, a counter p_j is associated with each column A_j . It represents the number of times A_j has been penalised (initially, $p_j = 0 \forall j \in J$). Let $\mathcal{C}$ denote a feasible cover obtained at the end of a given iteration. A utility function u_j is defined for each column A_j as $u_j = c_j / (1 + p_j)$. Let $\mathcal{C}^{\max} \subset \mathcal{C}$ denote the set of columns which are presented in the cover and which have the maximal utility. Then the attributes of each column $A_j \in \mathcal{C}^{\max}$ are updated as follows:

- $p_j = p_j + 1$.
- $c_j = (1 + \lambda)c_j$

Therefore, each time a column has a maximal utility, its cost is increased by 100%, thus making it less attractive. The parameter λ which represents the relative importance of the penalties is adjusted after experimentation. In our experiments, we have found that the value of 0.4 works well.

As it can be appreciated from the structure of the utility function, the procedure does not penalise ‘blindly’ the most costly columns, but takes into account the number of times the column has already been penalized. In this way, it tends to accept costly column if they seem inevitable in a good solution.

Speeding up the procedure

In order to speed up the procedure, we implemented a pre-processing phase that aims at reducing the instance size by removing some uninteresting columns. The pre-processing strategy is based on the following observation: when iterating the probabilistic greedy method, relatively few columns are chosen, at a given iteration, to be members of the set B of candidate columns. Of course, if the parameter s is fixed at a relatively large value then a larger number of columns would be chosen. The idea of the preprocessing procedure, is to fix s at some relatively large value (s_1), and to perform a restricted number of K_1 iterations. Each time a column is chosen to be member of B , it is immediately labelled. After performing the complete cycle of K_1 iterations, all unlabelled columns are *permanently* removed, and the search is

pursued with a reduced instance that contains only columns that have been chosen at least one time during the first K_1 iterations. The reader may have noticed that if the parameters s_1 and K_1 are relatively large (small), then few (many) columns would be removed. The pre-processing phase would be ineffective if only a very small number of columns are removed. On the contrary, one should be cautious about removing a large number of columns since this may lead to solutions of inferior quality. Hence, a good trade off must be reached.

A second idea, that we call ‘partial destruction’, has been implemented for accelerating the procedure. It consists in avoiding building a new solution from scratch at each new iteration of the greedy procedure. Instead, the best feasible solution found so far is considered and some of its columns are discarded. The columns that have been penalised in the last iteration are removed first, then additional columns are removed randomly, one at a time, until a fraction γ of the rows becomes uncovered. Our experimental study indicates that a value of 0.2 for γ yields good results. Hence, the probabilistic greedy procedure is restarted with the purpose of covering only a restricted subset of rows. The advantage of the partial destruction strategy is two-fold: it fastens the construction phase, and it intensifies the search of a new solution in the close neighbourhood of the most promising solutions.

Outline of the PROGRES method

Phase 1: Preprocessing

```
Repeat  $K_1$  times
  Set  $U = \emptyset$ 
  probabilistic_greedy( $s_1, c, U$ )
End (Repeat)
Set  $\mathcal{C}^*$  = best solution obtained so far
Remove permanently all unlabelled columns
End (Phase 1)
```

Phase 2: Iterating Greedy Search

```
Set  $U = \emptyset$ 
Repeat  $K_2$  times
  (Perturbation) Set  $\tilde{c}$  = Perturbed costs
  probabilistic_greedy( $s_2, \tilde{c}, U$ )
  (Penalisation) Set  $\mathcal{C}^{\max}$  = set of columns with maximal utility
  For all  $A_j \in \mathcal{C}^{\max}$  set  $c_j = (1 + \lambda)c_j$ 
  (Partial destruction) Set  $\mathcal{C}^*$  = best solution obtained so far
  Drop penalized columns from  $\mathcal{C}^*$ 
  Randomly drop columns from  $\mathcal{C}^*$  (until 100% of the rows are uncovered)
  Set  $U$  = set of uncovered rows
End (Repeat)
Output  $\mathcal{C}^*$  as the best solution found and stop
End (Phase 2)
```

Experimental results

Our experimental study aims at proving that the PROGRES method can yield a simple, fast, robust, and easy-to-code algorithm that produces high-quality solutions for large scale SCP instances. The test-bed we have used consists of 65 SCP instances drawn from the well-known Beasley's OR Library (Beasley³¹). These benchmark instances offer the significant advantage that almost all the SCP heuristics that have been developed during the last decade were tested

Table 1 Parameters setting

<i>Parameter</i>	<i>Value</i>
K_1	30
K_2	300
s_1	6
s_2	3
λ	0.4
γ	0.2

Table 2 Average excess over the best known solution: results on sets 4, 5, 6, A, and B

<i>Instances</i>	<i>m</i>	<i>n</i>	<i>Den.</i>	<i>BNS</i>	<i>G</i>		<i>PROGRES</i>			<i>B</i>		<i>H</i>		<i>BC</i>		<i>CFT</i>	
					<i>Sol.</i>	<i>Gap</i>	<i>Sol.</i>	<i>Time</i>	<i>Gap</i>	<i>Sol.</i>	<i>Gap</i>	<i>Sol.</i>	<i>Gap</i>	<i>Sol.</i>	<i>Gap</i>	<i>Sol.</i>	<i>Gap</i>
① SCP41	200	1000	2%	429*	463	7.93	430	8.80	0.23	429	0.00	429	0.00	429	0.00	429	0.00
SCP42	200	1000	2%	512*	582	13.67	514	8.60	0.39	512	0.00	512	0.00	512	0.00	512	0.00
SCP43	200	1000	2%	516*	589	14.15	519	11.30	0.58	516	0.00	516	0.00	516	0.00	516	0.00
SCP44	200	1000	2%	494*	548	10.93	498	18.30	0.81	495	0.20	494	0.00	494	0.00	494	0.00
SCP45	200	1000	2%	512*	577	12.70	514	8.20	0.39	512	0.00	512	0.00	512	0.00	512	0.00
SCP46	200	1000	2%	560*	615	9.82	564	8.60	0.71	561	0.18	560	0.00	560	0.00	560	0.00
SCP47	200	1000	2%	430*	476	10.70	434	8.00	0.93	430	0.00	430	0.00	430	0.00	430	0.00
SCP48	200	1000	2%	492*	533	8.33	493	8.60	0.20	493	0.20	493	0.20	492	0.00	492	0.00
SCP49	200	1000	2%	641*	680	6.08	648	13.00	1.09	641	0.00	641	0.00	641	0.00	641	0.00
SCP410	200	1000	2%	514*	556	8.17	516	13.10	0.39	514	0.00	514	0.00	514	0.00	514	0.00
<i>Average gap</i>						10.25			0.57		0.06		0.02		0.00		0.00
SCP51	200	2000	2%	253*	279	10.28	258	16.20	1.98	255	0.79	254	0.40	253	0.00	253	0.00
SCP52	200	2000	2%	302*	338	11.92	307	14.90	1.66	304	0.66	306	1.32	302	0.00	302	0.00
SCP53	200	2000	2%	226*	246	8.85	228	10.20	0.88	226	0.00	226	0.00	228	0.88	226	0.00
SCP54	200	2000	2%	242*	265	9.50	244	10.40	0.83	242	0.00	242	0.00	242	0.00	242	0.00
SCP55	200	2000	2%	211*	236	11.85	212	13.30	0.47	211	0.00	211	0.00	211	0.00	211	0.00
SCP56	200	2000	2%	213*	235	10.33	214	15.40	0.47	213	0.00	213	0.00	213	0.00	213	0.00
SCP57	200	2000	2%	293*	326	11.26	295	18.00	0.68	294	0.34	294	0.34	293	0.00	293	0.00
SCP58	200	2000	2%	288*	323	12.15	290	16.00	0.69	288	0.00	288	0.00	288	0.00	288	0.00
SCP59	200	2000	2%	279*	312	11.83	280	14.90	0.36	279	0.00	279	0.00	279	0.00	279	0.00
SCP510	200	2000	2%	265*	293	10.57	267	13.10	0.75	265	0.00	265	0.00	265	0.00	265	0.00
<i>Average gap</i>						10.85			0.88		0.18		0.21		0.09		0.00
SCP61	200	1000	5%	138*	150	8.70	140	5.80	1.45	141	2.17	141	2.17	138	0.00	138	0.00
SCP62	200	1000	5%	146*	160	9.59	147	4.80	0.68	146	0.00	146	0.00	146	0.00	146	0.00
SCP63	200	1000	5%	145*	159	9.66	146	4.50	0.69	145	0.00	145	0.00	145	0.00	145	0.00
SCP64	200	1000	5%	131*	141	7.63	131	5.00	0.00	131	0.00	131	0.00	131	0.00	131	0.00
SCP65	200	1000	5%	161*	176	9.32	162	5.00	0.62	162	0.62	162	0.62	161	0.00	161	0.00
<i>Average gap</i>						8.98			0.69		0.56		0.56		0.00		0.00
SCPA1	300	3000	2%	253*	270	6.72	255	48.30	0.79	255	0.79	254	0.40	253	0.00	253	0.00
SCPA2	300	3000	2%	252*	273	8.33	253	33.80	0.40	256	1.59	253	0.40	252	0.00	252	0.00
SCPA3	300	3000	2%	232*	252	8.62	234	33.60	0.86	234	0.86	234	0.86	232	0.00	232	0.00
SCPA4	300	3000	2%	234*	258	10.26	236	38.80	0.85	235	0.43	234	0.00	234	0.00	234	0.00
SCPA5	300	3000	2%	236*	258	9.32	238	37.90	0.85	237	0.42	236	0.00	236	0.00	236	0.00
<i>Average gap</i>						8.65			0.75		0.82		0.33		0.00		0.00
SCPB1	300	3000	5%	69*	71	2.90	69	28.20	0.00	70	1.45	69	0.00	69	0.00	69	0.00
SCPB2	300	3000	5%	76*	77	1.32	76	26.50	0.00	77	1.32	76	0.00	76	0.00	76	0.00
SCPB3	300	3000	5%	80*	86	7.50	80	31.10	0.00	80	0.00	81	1.25	80	0.00	80	0.00
SCPB4	300	3000	5%	79*	89	12.66	79	40.80	0.00	80	1.27	79	0.00	79	0.00	79	0.00
SCPB5	300	3000	5%	72*	78	8.33	72	31.10	0.00	72	0.00	72	0.00	72	0.00	72	0.00
<i>Average gap</i>						6.54			0.00		0.81		0.25		0.00		0.00

on these instances. These instances are divided into 11 classes. Each class is characterised by a number of instances (5 or 10), a number of rows (ranging from 200 to 1000), a number of columns (ranging from 1000 to 10 000), and a density (ranging from 2 to 20%).

The PROGRES method was coded in Microsoft Visual C++ (version 5.0). All our experiments were obtained on a Pentium III 800 MHz PC. We used the parameter setting displayed in Table 1. These parameters were fixed on the basis of preliminary experimentation.

We compared the performance of the PROGRES method with the (simple) greedy method (G) along with the following four heuristics:

- B: the Lagrangian heuristic of Beasley¹⁸
- BC: the genetic algorithm of Beasley and Chu²³
- H: the Lagrangian heuristic of Haddadi³²
- CFT: the Lagrangian heuristic of Caprara *et al*²¹

These methods are not only recent, but also perform extremely well. In particular, BC and CFT are currently considered as the best existing procedures. Tables 2 and 3 summarise the percentage deviation from the best known solution (proven optimal solutions are indicated with an asterisk). In these tables the entries have the following meaning:

Instance : the name of the instance;
m : number of columns;

Table 3 Average excess over the best known solution: results on sets C, D, E, F, G, and H

Instances	<i>m</i>	<i>n</i>	Den.	BNS	<i>G</i>		<i>PROGRES</i>			<i>B</i>		<i>H</i>		<i>BC</i>		<i>CFT</i>	
					Sol.	Gap	Sol.	Time	Gap	Sol.	Gap	Sol.	Gap	Sol.	Gap	Sol.	Gap
SCPC1	400	4000	2%	227*	258	13.66	230	72.90	1.32	230	1.32	228	0.44	227	0.00	227	0.00
SCPC2	400	4000	2%	219*	251	14.61	220	69.20	0.46	223	1.83	223	1.83	219	0.00	219	0.00
SCPC3	400	4000	2%	243*	276	13.58	247	100.60	1.65	251	3.29	245	0.82	243	0.00	243	0.00
SCPC4	400	4000	2%	219*	247	12.79	220	77.20	0.46	224	2.28	223	1.83	219	0.00	219	0.00
SCPC5	400	4000	2%	215*	233	8.37	216	75.60	0.47	217	0.93	216	0.47	215	0.00	215	0.00
Average gap						12.66			0.87		1.93		1.08		0.00		0.00
SCPD1	400	4000	5%	60*	72	20.00	60	45.10	0.00	61	1.67	61	1.67	60	0.00	60	0.00
SCPD2	400	4000	5%	66*	74	12.12	66	74.70	0.00	68	3.03	66	0.00	66	0.00	66	0.00
SCPD3	400	4000	5%	72*	83	15.28	72	84.70	0.00	75	4.17	73	1.39	72	0.00	72	0.00
SCPD4	400	4000	5%	62*	71	14.52	62	65.10	0.00	64	3.23	62	0.00	62	0.00	62	0.00
SCPD5	400	4000	5%	61*	69	13.11	61	76.70	0.00	62	1.64	62	1.64	61	0.00	61	0.00
Average gap						15.01			0.00		2.75		0.94		0.00		0.00
SCPE1	500	5000	10%	29	32	10.34	29	125.80	0.00	29	0.00	29	0.00	29	0.00	29	0.00
SCPE2	500	5000	10%	30	36	20.00	30	134.20	0.00	32	6.67	32	6.67	30	0.00	30	0.00
SCPE3	500	5000	10%	27	30	11.11	27	110.40	0.00	28	3.70	28	3.70	27	0.00	27	0.00
SCPE4	500	5000	10%	28	34	21.43	28	129.70	0.00	30	7.14	29	3.57	28	0.00	28	0.00
SCPE5	500	5000	10%	28	31	10.71	28	112.10	0.00	28	0.00	28	0.00	28	0.00	28	0.00
Average gap						14.72			0.00		3.50		2.79		0.00		0.00
SCPF1	500	5000	20%	14	18	28.57	14	89.60	0.00	15	7.14	14	0.00	14	0.00	14	0.00
SCPF2	500	5000	20%	15	19	26.67	15	79.60	0.00	16	6.67	15	0.00	15	0.00	15	0.00
SCPF3	500	5000	20%	14	18	28.57	14	106.40	0.00	15	7.14	15	7.14	14	0.00	14	0.00
SCPF4	500	5000	20%	14	16	14.29	15	103.20	7.14	15	7.14	15	7.14	14	0.00	14	0.00
SCPF5	500	5000	20%	13	16	23.08	13	110.70	0.00	14	7.69	14	7.69	13	0.00	13	0.00
Average gap						24.23			1.43		7.16		4.40		0.00		0.00
SCPG1	1000	10 000	2%	176	194	10.23	180	465.80	2.27	184	4.55	181	2.84	176	0.00	176	0.00
SCPG2	1000	10 000	2%	154	173	12.34	155	466.80	0.65	163	5.84	160	3.90	155	0.65	154	0.00
SCPG3	1000	10 000	2%	166	182	9.64	167	536.70	0.60	174	4.82	174	4.82	166	0.00	166	0.00
SCPG4	1000	10 000	2%	168	180	7.14	170	598.20	1.19	176	4.76	177	5.36	168	0.00	168	0.00
SCPG5	1000	10 000	2%	168	184	9.52	170	604.80	1.19	175	4.17	173	2.98	168	0.00	168	0.00
Average gap						9.77			1.18		4.83		3.98		0.13		0.00
SCPH1	1000	10 000	5%	63	72	14.29	64	1123.60	1.59	68	7.94	66	4.76	64	1.59	63	0.00
SCPH2	1000	10 000	5%	63	71	12.70	64	985.40	1.59	66	4.76	67	6.35	64	1.59	63	0.00
SCPH3	1000	10 000	5%	59	66	11.86	60	1459.00	1.69	65	10.17	62	5.08	59	0.00	59	0.00
SCPH4	1000	10 000	5%	58	65	12.07	59	782.60	1.72	63	8.62	61	5.17	58	0.00	58	0.00
SCPH5	1000	10 000	5%	55	60	9.09	56	695.20	1.82	60	9.09	56	1.82	55	0.00	55	0.00
Average gap						12.00			1.68		8.12		4.64		0.63		0.00
Average gap (over all instances)						11.9			0.72		2.36		1.49		0.07		0.00

n : number of rows;
 Den. : density;
 BNS : value of the best known solution;
 Sol. : value of the solution provided by the heuristic;
 Gap : percentage deviation from the best known solution [$100 \times ((\text{sol} - \text{BNS})/\text{BNS})$].
 Time : the total CPU time in seconds on a Pentium III 800 MHz PC.

Tables 2 and 3 clearly indicate that the PROGRES method dramatically outperforms the greedy heuristic. Hence, the average percentage deviation of the greedy method being 11.9%, whereas it is only 0.73% for PROGRES. Moreover, the experiments confirm the erratic behaviour of the greedy method since the percentage deviations are ranging between 1.32 and 28.57%. This clearly indicates the pertinence of the implemented enhancements. More importantly, we observe that PROGRES seems to perform remarkably well on all problem classes. Indeed, we see that its average percentage deviation is smaller than the one obtained with heuristics B and H (2.36 and 1.49%, respectively). However, PROGRES is dominated by the very effective methods BC and CFT, but the lack of performance is surprisingly small. Indeed, for 20 instances out of 65 (30.77%) PROGRES provides a solution which is as good as the best known one, and in most cases (76.92%) it yields a percentage deviation which is strictly less than 1%. Moreover, the proposed method is robust, since the largest percentage deviation exceeded 2% for only two instances out of 65. The largest deviation being 7.14% and was obtained for an instance where PROGRES proposes a solution of value 15 whereas the best one is 14.

Finally, we find that the running time is relatively modest and averages at 156.70 s. The largest CPU time has been reached for the instance SCPH3 (1000 rows and 10 000 columns) which required 1459.00 s. We compare this to the single run of the BC heuristic that requires an average CPU time of 2500.00 s on a Silicon Graphics workstation.

Conclusion

In this paper, we have presented an improved probabilistic greedy approach and used it to build a new heuristic for the SCP. Our approach incorporates diversification schemes along with a learning strategy into the standard greedy method. The proposed approach exhibits several interesting characteristics: it is simple, quite fast, easy-to-code, robust, and provides solutions that are competitive with those obtained with the best existing methods. Although more powerful special purpose methods are available for the SCP (BC and CFT), the proposed approach constitutes a particularly interesting alternative for a user who is not willing to invest a significant amount of time and efforts in implementing a highly sophisticated technique. Furthermore, the main ideas underlying our approach can be tailored and used to

tackle a wide variety of combinatorial optimisation problems.

Acknowledgements—The authors would like to thank an anonymous referee for several suggestions which improved the paper.

References

- 1 Nemhauser GL and Wolsey LA (1988). *Integer and Combinatorial Optimization*. Wiley: Chichester.
- 2 Johnson DS and McGeoch LA (1997). The traveling salesman problem: a case study in local search. In: Aarts E and Lenstra JK (eds). *Local Search in Combinatorial Optimization*. Wiley: Chichester, pp 215–310.
- 3 Feo TA and Resende MGC (1989). A probabilistic heuristic for a computationally difficult set covering problem. *Opns Res Lett* **67**: 67–71.
- 4 Feo TA and Resende MGC (1995). Greedy randomized adaptive search procedures. *J Global Opt* **6**: 109–133.
- 5 Hoffman K and Padberg M (1993). Solving airline crew-scheduling problems by branch-and-cut. *Mngt Sci* **39**: 657–682.
- 6 Paías A and Paixao J (1993). State space relaxation for the set covering problem related to bus driver scheduling. *Eur J Opl Res* **71**: 303–316.
- 7 Desrosiers J, Dumas Y, Solomon MM and Soumis F (1995). Time constrained routing and scheduling. In: Ball MO, Magnanti TL, Monma CL and Nemhauser GL (eds). *Handbooks in Operations Research and Management Science Volume 8: Network Routing*. North-Holland: Amsterdam, pp 35–139.
- 8 Current J and O’Kelly M (1992). Locating emergency warning sirens. *Dec Sci* **23**: 221–234.
- 9 Beasley JE (1987). An algorithm for the set covering problem. *Eur J Opl Res* **31**: 85–93.
- 10 Beasley JE and Jörnsten K (1992). Enhancing an algorithm for set covering problems. *Eur J Opl Res* **58**: 293–300.
- 11 El Darzi E and Mitra G (1992). Solution of set-covering and set-partitioning problems using assignment relaxations. *J Opl Res Soc* **43**: 483–493.
- 12 Nobili P and Sassano A (1992). A separation routine for the set covering polytope. *Proceedings of the 2nd IPCO Conference*. Carnegie-Mellon University Press: Pittsburgh, PA, pp 201–219.
- 13 Rushmeier R and Nemhauser GL (1993). Experiments with parallel branch-and-bound algorithms for the set covering problem. *Opns Res Lett* **13**: 227–286.
- 14 Harche F and Thompson GL (1994). The column subtraction algorithm: an exact method for solving weighted set covering, packing, and partitioning problems. *Comput Opns Res* **21**: 698–706.
- 15 Balas E and Carrera MC (1996). A dynamic subgradient-based branch and bound procedure for set covering. *Opns Res* **44**: 875–890.
- 16 Balas E, Ceria S and Cornuéjols G (1996). Mixed 0–1 programming by lift-and-project in a branch-and-cut framework. *Mngt Sci* **42**: 1229–1246.
- 17 Chvátal V (1979). A greedy heuristic for the set covering problem. *Math Opns Res* **4**: 233–235.
- 18 Beasley JE (1990). A lagrangian heuristic for set covering problems. *Naval Res Log Q* **37**: 151–164.
- 19 Lorena LAN and Lopes FB (1994). A surrogate heuristic for set covering problems. *Eur J Opl Res* **79**: 138–150.
- 20 Ceria S, Nobili P and Sassano A (1998). A lagrangian-based heuristic for large-scale set covering problems. *Math Prog* **81**: 215–228.

- 21 Caprara A, Fischetti M and Toth P (1999). A heuristic method for the set covering problem. *Opns Res* **47**: 730–743.
- 22 Jacobs LW and Brusco ML (1995). Note: a local-search heuristic for large set covering problems. *Naval Res Log Q* **42**: 1129–1140.
- 23 Beasley JE and Chu PC (1996). A genetic algorithm for the set covering problem. *Eur J Opl Res* **94**: 392–404.
- 24 Al-Sultan KS, Hussain MF and Nizami JS (1996). A genetic algorithm for the set covering problem. *J Opl Res Soc* **47**: 702–709.
- 25 Lorena LAN and Lopes FB (1997). Genetic algorithms applied to computationally difficult set covering problems. *J Opl Res Soc* **48**: 440–445.
- 26 Resende MGC (1998). Greedy randomized search procedure (GRASP). AT&T Labs Research Technical Report: 98.41.1.
- 27 Storer RH, Wu SD and Vaccari R (1992). New search spaces for sequencing problems with application to job shop scheduling. *Mngt Sci* **38**: 1495–1509.
- 28 Charon I and Hudry O (1993). The noising method: a new method for combinatorial optimization. *Opns Res Lett* **14**: 133–137.
- 29 Codenotti B, Manzini G, Margara L and Resta G (1996). Perturbation: an efficient technique for the solution of very large instances of the Euclidean TSP. *INFORMS J Comput* **8**: 125–133.
- 30 Tsang E and Voudouris C (1999). Guided local search and its application to the travelling salesman problem. *Eur J Opl Res* **113**: 469–499.
- 31 Beasley JE (1990). OR-Library: distributing test problems by electronic mail. *J Opl Res Soc* **41**: 1069–1072.
- 32 Haddadi S (1997). Simple lagrangian heuristic for the set covering problem. *Eur J Opl Res* **97**: 200–204.

*Received March 2001;
accepted November 2001 after one revision*